



Instituto Tecnológico de Puerto Rico Recinto de San Juan

DOCUMENTACIÓN DEL PROGRAMA PERSONAL

Versiones 1.0, 2.0 y 3.0

Plantilla para acompañar el Laboratorio 3 - Semana 4

Estudiante	Tairon A. Jaime Martinez
Repositorio	TESI2105_Programa_Evolutivo_JaimeTairon
Nombre del programa	
Fecha	8 de Septiembre de 2026

1. DESCRIPCIÓN GENERAL Y CONTINUIDAD

Describe brevemente la necesidad que atiende tu programa y explica cómo las versiones anteriores preparan el punto de partida para v3.0. No repitas las respuestas del documento del laboratorio; resume la continuidad en 4 a 6 oraciones.

Se reorganizó el código de la v2.0 separando el flujo en tareas específicas como la validación de tiempo, cálculo de puntos y gestión de la tienda. El código original era un bloque lineal, ahora se convirtió en un programa modular donde cada función tiene su responsabilidad única, facilitando su estructura. La lógica central del minijuego se mantiene en una estructura mientras dentro de la función main(), lo cual coordina la ejecución de las demás funciones. Esto permite que el programa sea más escalable al separar la lógica de negocio de la interfaz del usuario.

2. RESUMEN DE LAS VERSIONES ANTERIORES

Versión	Base técnica	Cómo se refleja en mi programa
v1.0	Programa secuencial con entradas, variables, constantes, operadores y salidas.	En la versión v1.0 lo único que hace es solicitarle el nombre de usuario, la clase del personaje y los segundos jugados, usando una constante para calcular los puntos y mostrándote el resultado. No solo eso se podía poner números negativos.
v2.0	El mismo programa incorpora decisiones y ciclos.	En esta versión se añadió una validación de que los segundos no fueran negativos y un loop para repetir la entrada hasta que el usuario escribiera un valor mayor que cero.

Este resumen no sustituye la documentación entregada en la Semana 3; únicamente establece continuidad para la nueva versión.

3. VERSIÓN 3.0 - ORGANIZACIÓN MEDIANTE FUNCIONES

Explica en 3 a 5 oraciones cómo reorganizamos el código de v2.0 para crear v3.0 mediante funciones con responsabilidades claras.

Se reorganizó el código v2.0 dividiendo el flujo en tareas específicas como en la validación de tiempo, cálculo de puntos y gestión de la tienda. El código original, era un bloque lineal, ahora es un programa modular donde cada función tiene una responsabilidad única, facilitando su lectura. La lógica principal del minijuego se mantiene en una estructura mientras dentro de la función `main()`, la cual coordina la ejecución de las demás funciones. Esto hace que el programa sea más escalable al separar la lógica de negocio de la interfaz del usuario.

4. FUNCIONES CREADAS EN V3.0

Completa una fila por cada función creada. Debes documentar al menos tres funciones.

Función	Propósito	Parámetros	Retorno o acción
<code>validar_tiempo()</code>	Solicita y valida la entrada del usuario asegurando que sea mayor a 0.	Ninguno	Retorna el valor entero ingresado por el usuario.
<code>calcular_puntos()</code>	Calcula los puntos finales multiplicando tiempo, clase y boost.	Segundos, clase y multiplicador	Retorna el valor calculado de los puntos.
<code>mostrar_tienda()</code>	Muestra opciones de compra y gestionar el saldo del usuario.	<code>dinero_actual</code>	Retorna el dinero restante y el nuevo valor del multiplicador.

5. FUNCIÓN INCORPORADA DE PYTHON

Función incorporada (built-in) : max()

Dónde se utiliza: Dentro de la función calcular_puntos().

Propósito dentro del programa: Asegurar que el resultado final de los puntos nunca sea un número negativo, devolviendo 0 en caso de que ocurriera algún error en el cálculo, garantizando así la integridad de los datos.

6. INTEGRACIÓN DE LA LÓGICA DE V2.0

Decisión que se conserva: La verificación de la clase del personaje (Mago o Guerrero) para asignar el bono de puntos.

Función donde se encuentra la decisión: calcular_puntos()>

Ciclo que se conserva: El ciclo while para mantener el juego activo y permitir múltiples rondas.

Función o parte del flujo donde se ejecuta el ciclo: main().

7. PRUEBAS DE FUNCIONAMIENTO DE V3.0

Registra al menos dos pruebas realizadas después de modularizar el programa.

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido
1	Ingresar "-5" en segundos.	Mostrar mensaje de error: "Error: no se puede poner un número menor que 1" y volver a pedir el dato.	El programa mostró el error correctamente y solicitó el dato nuevamente.
2	Jugar una ronda con 10 segundos siendo "Guerrero" (sin multiplicador).	10 puntos (10 * 1 * 1).	El programa sumó 10 puntos correctamente.
3	Comprar multiplicador y jugar 10 segundos siendo "Mago".	40 puntos (10 * 1 * 2 [Mago] * 2 [Multiplicador]).	El programa calculó 40 puntos correctamente.

8. REFLEXIÓN DE EVOLUCIÓN

¿Por qué v3.0 es más fácil de leer, reutilizar o mantener que v2.0? Explica en 4 a 6 oraciones y menciona al menos una función de tu programa como ejemplo.

La versión 3.0 es significativamente más fácil de mantener que la v2.0 porque el código está modularizado. Por ejemplo, si decido cambiar la fórmula de puntos, solo debo modificar la función calcular_puntos() sin tener que buscar y editar bloques dentro de un ciclo gigante. Esto evita errores accidentales en otras partes del programa. Además, al usar funciones con parámetros claros, el código se vuelve más legible y reutilizable, permitiendo entender la lógica del juego con solo leer los nombres de las funciones. La modularidad también permite probar cada componente de forma aislada, lo cual es mucho más eficiente que probar todo el programa completo a la vez.

Modularizar significa dividir un programa grande y complejo en piezas más pequeñas, independientes y organizadas, llamadas "módulos".

9. VALIDACIÓN

Etiquetas verificadas en el repositorio: X v1.0 X v2.0 X v3.0

Archivo PDF en documentacion/: Documentacion_V1_V2_V3.pdf

URL del repositorio: https://github.com/Mofongo300/TESI2105_Programa_Evolutivo_TaironJaime

Nota: Esta plantilla no requiere diagramas de flujo ni diagramas modulares. La evidencia se concentra en la organización funcional del código, su documentación y las pruebas realizadas.